Yes — now I can see exactly where the remaining confusion is. Your **big-picture direction is right**, but the order of operations in your mental model is slightly off.

The most important correction is:

LPA* does NOT first call A* to create a complete path and then repair it.\boxed{\text{LPA* does NOT first call A* to create a complete path and then repair it.}}

Instead:

ComputeShortestPath() itself IS the LPA* search/repair procedure.\boxed{\texttt{ComputeShortestPath()} \text{ itself IS the LPA* search/repair procedure.}}

On the **first run**, that procedure behaves like A*. After edge-cost changes, the *same machinery* repairs the previous solution incrementally. The paper explicitly says the first `ComputeShortestPath()` expands the same vertices as A* in the same order.

So don't imagine:

LPA*
  ↓
call A*
  ↓
get path
  ↓
then update vertices

Instead imagine:

LPA*

Initialize g/rhs
    ↓
inconsistent vertices enter OPEN
    ↓
ComputeShortestPath()
    ↓
repair/propagate values using
A*-like priority
    ↓
goal becomes sufficiently settled
    ↓
shortest path can be extracted
    ↓

edge changes later
↓
affected vertices become inconsistent
↓
ComputeShortestPath() repairs them

That distinction will make everything click.

1. LPA* — final notebook flow

Start with the equations.

For **forward LPA***:

rhs(sstart)=0\boxed{rhs(s_{start})=0}

and for every other state uu:

rhs(u)=minp∈Pred(u)[g(p)+c(p,u)]\boxed{ rhs(u) = \min_{p\in Pred(u)} [g(p)+c(p,u)] }

The key is:

m(u)=min(g(u),rhs(u))m(u)=\min(g(u),rhs(u)) key(u)=[m(u)+h(u,sgoal), m(u)]\boxed{ key(u)= [m(u)+h(u,s_{goal}),\;m(u)] }

The priority queue contains inconsistent vertices:

g(u)≠rhs(u)\boxed{g(u)\neq rhs(u)}

and the smallest key is processed first.

LPA* FLOW MAP

Write this in your notebook:

LPA* = Forward incremental A*

SEARCH DIRECTION:
START → GOAL

[1] INITIALIZE

---

for every state s:

    g(s)   = ∞
    rhs(s) = ∞

rhs(start) = 0

OPEN = empty priority queue

Since:

    g(start) ≠ rhs(start)

insert START into OPEN
with CalculateKey(start)

↓

[2] COMPUTE SHORTEST PATH
---

while:

    goal is still inconsistent

      OR

    there exists an OPEN state
    with better priority than goal:

        pop u with smallest KEY

        CASE A:

        g(u) > rhs(u)

        means:

        "new information says

u can be cheaper"

therefore:

g(u) = rhs(u)

u becomes consistent

because g(u) changed:

UpdateVertex(v)

for every:

v ∈ Succ(u)

CASE B:

g(u) < rhs(u)

means:

"old g(u) was too cheap /
no longer valid"

therefore:

g(u) = ∞

then repair:

u itself
+
every v ∈ Succ(u)

using UpdateVertex()

↓

[3] GOAL SETTLED

When ComputeShortestPath stops:

we have enough correct g-values
to obtain shortest path

START → GOAL

↓

[4] WAIT FOR EDGE-COST CHANGE

Suppose edge:

u → v

changes cost.

In forward LPA*:

rhs(v) depends on:

g(u) + c(u,v)

therefore:

UpdateVertex(v)

If v becomes inconsistent:

put/update v in OPEN

↓

[5] CALL ComputeShortestPath AGAIN

Do NOT restart search.

Reuse previous g/rhs values.

Repair only affected/relevant
inconsistent states.

↓

Repeat whenever graph changes.

This matches the paper's actual flow: initialize, run `ComputeShortestPath`, wait for edge-cost changes, call `UpdateVertex(v)` on the directly affected endpoint, then recompute.

---

2. What exactly does `UpdateVertex(u)` do?

This was mixed up in your explanation.

`UpdateVertex()` does **not normally set g(u)=rhs(u)g(u)=rhs(u)**.

That happens inside:

ComputeShortestPath()\boxed{\texttt{ComputeShortestPath()}}

`UpdateVertex()` instead asks:

> "According to my neighbors, what should uu's value be, and should uu be in OPEN?"

For LPA*:

UpdateVertex(u):

if u != start:

rhs(u)
=

```
    min over p ∈ Pred(u)
    [g(p) + c(p,u)]

  if u already in OPEN:

    remove old entry

  if g(u) != rhs(u):

    insert u into OPEN
    with CalculateKey(u)

  else:

    leave it outside OPEN
```

That is essentially the paper's `UpdateVertex()` procedure.

So separate these two operations in your head:

UpdateVertex=recalculate rhs+manage OPEN\boxed{ \texttt{UpdateVertex} = \text{recalculate }rhs + \text{manage OPEN} }

while:

ComputeShortestPath=actually change g+propagate repairs\boxed{ \texttt{ComputeShortestPath} = \text{actually change }g + \text{propagate repairs} }

That is important.

---

3. Why successors are affected in LPA*

For LPA*:

rhs(v)=minp∈Pred(v)[g(p)+c(p,v)].rhs(v) = \min_{p\in Pred(v)} [g(p)+c(p,v)].

Suppose:

g(u)g(u)

changes.

Who has uu as a predecessor?

Its:

Succ(u).\boxed{Succ(u)}.

Therefore:

g(u) changes⇒UpdateVertex(Succ(u))\boxed{ g(u)\text{ changes} \Rightarrow UpdateVertex(Succ(u)) }

So information propagates:

START→GOAL\boxed{ START\rightarrow GOAL }

which resembles forward value propagation.

---

4. Where exactly is A* inside LPA*?

You gave ordinary A* pseudocode correctly enough, but **we don't embed that pseudocode as a separate function inside LPA***.

Instead, LPA* borrows the **search philosophy** of A*.

A* has:

f(n)=g(n)+h(n)\boxed{f(n)=g(n)+h(n)}

and chooses the smallest ff.

LPA* has:

m(n)=min(g(n),rhs(n))m(n)=\min(g(n),rhs(n))

and:

k1=m+h\boxed{ k_1=m+h } k2=m\boxed{ k_2=m }

then chooses smallest:

[k1,k2][k_1,k_2]

lexicographically.

So the mapping is:

A*

g(n)
+
h(n)
=
f(n)

LPA*

min(g,rhs)
+
h
=
k1

and

min(g,rhs)
=
k2

Therefore the A* contribution is:

heuristic-guided priority expansion\boxed{ \text{heuristic-guided priority expansion} }

plus the A*-style interpretation of gg.

But LPA* adds:

rhs\boxed{rhs}

and:

memory across searches\boxed{\text{memory across searches}}

and:

local inconsistency repair.\boxed{\text{local inconsistency repair}}.

The paper explicitly says LPA*'s first search behaves exactly like A* under the stated conditions, while later searches reuse stored values.

So notebook:

LPA∗=A∗+rhs+incremental memory\boxed{ LPA* = A^* + rhs + incremental\ memory }

Conceptually, that's excellent.

---

5. Now D* Lite

Your sentence:

> "D* Lite just reverses LPA*."

is **mostly correct**.

The paper explicitly says D* Lite swaps start and goal and reverses the search direction in the pseudocode.

But also add:

+moving robot handling using km\boxed{+\text{moving robot handling using }k_m}

So write:

D∗Lite=reverse LPA∗+moving start\boxed{ D^*Lite = reverse\ LPA^* + moving\ start }

---

6. D* Lite equations

Now initialization happens at the **goal**:

rhs(goal)=0\boxed{rhs(goal)=0}

All:

g(s)=rhs(s)=∞g(s)=rhs(s)=\infty

initially.

For any non-goal state uu:

rhs(u)=minv∈Succ(u)[c(u,v)+g(v)]\boxed{ rhs(u) = \min_{v\in Succ(u)} [c(u,v)+g(v)] }

This is exactly the backward cost-to-go form.

And the key:

m(u)=min(g(u),rhs(u))m(u)=\min(g(u),rhs(u)) key(u)=[m(u)+h(sstart,u)+km, m(u)]\boxed{ key(u) = [m(u)+h(s_{start},u)+k_m,\; m(u)] }

---

7. D* Lite — final notebook flow

Write this one immediately after LPA*:

D* LITE = Reverse LPA*
+ Moving Robot

SEARCH DIRECTION:

GOAL → CURRENT START

[1] INITIALIZE
---

for every state s:

g(s) = ∞
rhs(s) = ∞

rhs(goal) = 0

km = 0

OPEN = empty

Since:

g(goal) ≠ rhs(goal)

insert GOAL into OPEN

↓

[2] COMPUTE SHORTEST PATH
---

while:

current START is inconsistent

OR

OPEN contains a state
with better priority
than current start:

pop smallest-key u

if stored key is outdated:

recalculate key
reinsert u

CASE A:

g(u) > rhs(u)

new/better route found

set:

g(u) = rhs(u)

because g(u) changed:

UpdateVertex(p)

for every:

p ∈ Pred(u)

CASE B:

g(u) < rhs(u)

old g(u) too optimistic

set:

g(u) = ∞

then update:

u
+
Pred(u)

↓

[3] CURRENT START IS SETTLED

Now the robot chooses:

next
=
argmin over v ∈ Succ(start)

[c(start,v) + g(v)]

↓

[4] MOVE ROBOT
---

start = next

physically move robot

↓

[5] SENSE ENVIRONMENT

Did edge costs change?

NO
↓

continue using current
goal-distance information

YES
↓

update km

km =
km + h(last_start, new_start)

update changed edge costs

For changed edge:

u → v

D* Lite directly updates:

UpdateVertex(u)

because:

rhs(u)
depends on:

c(u,v) + g(v)

↓

[6] CALL ComputeShortestPath AGAIN

Do not restart whole search.

Repair only inconsistent
relevant region.

↓

[7] MOVE AGAIN
---

choose:

argmin [c(start,v)+g(v)]

move

sense

repair if needed

↓

repeat until:

start = goal

That is the full high-level D* Lite algorithm from the paper's Figure 3.

---

8. Your misunderstanding: "After creating path, update only the path"

This part is **not correct**.

D* Lite does **not** say:

> Only update nodes belonging to the previous best path.

Instead:

update states whose consistency may be affected\boxed{ \text{update states whose consistency may be affected} }

and the priority queue decides which inconsistent states are relevant enough to process.

The affected region may overlap the old path, but it can also include states off the path.

The paper's motivation specifically says only some changed goal distances are relevant for replanning and that D* Lite aims to identify and update those efficiently.

So don't write:

repair path nodes only

Write:

repair affected inconsistent states
needed for current shortest path

---

9. The perfect direction comparison

This one is notebook gold.

Suppose:

u→v.u\rightarrow v.

LPA*

rhs(v)=minp∈Pred(v)[g(p)+c(p,v)]rhs(v) = \min_{p\in Pred(v)} [g(p)+c(p,v)]

therefore if:

c(u,v)c(u,v)

changes:

v is directly affected\boxed{v\text{ is directly affected}}

and propagation moves through:

Succ.Succ.

D* Lite

rhs(u)=minv∈Succ(u)[c(u,v)+g(v)]rhs(u) = \min_{v\in Succ(u)} [c(u,v)+g(v)]

therefore if:

c(u,v)c(u,v)

changes:

u is directly affected\boxed{u\text{ is directly affected}}

and propagation moves through:

Pred.Pred.

So:

	LPA*	D* Lite
boundary	START	GOAL
search	S → G	G → S
rhs uses	Pred	Succ
g change affects	Succ	Pred
edge u→v changes	update v	update u

This single table explains why you're calling D* Lite “reverse LPA*”.

---

10. Full pseudo-equation roadmap

If you want the **smallest cheat-sheet form**, copy exactly this:

==========
LPA*
==========

FORWARD:
START → GOAL

Init:

g(s)=rhs(s)=∞

rhs(start)=0

OPEN={start}

rhs(u):

min_{p∈Pred(u)}
[g(p)+c(p,u)]

consistent:

g(u)=rhs(u)

inconsistent:

g(u)≠rhs(u)
→ OPEN

key:

m=min(g,rhs)

k=[m+h(u,goal), m]

ComputeShortestPath:

pop min-key u

if g>rhs:
  g=rhs
  UpdateVertex(Succ(u))

else if g<rhs:
  g=∞
  UpdateVertex(u + Succ(u))

edge u→v changes:

UpdateVertex(v)

Repeat repair.
NO restart.

==========
D* LITE
==========

BACKWARD:
GOAL → CURRENT START

Init:

g(s)=rhs(s)=∞

rhs(goal)=0

km=0

OPEN={goal}

rhs(u):

min_{v∈Succ(u)}
[c(u,v)+g(v)]

consistent:

g=rhs

inconsistent:

g≠rhs
→ OPEN

key:

m=min(g,rhs)

k=[
m+h(current_start,u)+km,
m
]

ComputeShortestPath:

pop min-key u

if g>rhs:
    g=rhs
    UpdateVertex(Pred(u))

else if g<rhs:
    g=∞
    UpdateVertex(u + Pred(u))

when current start settled:

next =
argmin_{v∈Succ(start)}
[c(start,v)+g(v)]

MOVE

sense edge changes

if edge u→v changes:

```
    km += h(last_start,
            current_start)

    UpdateVertex(u)

    ComputeShortestPath()
```

repeat:

MOVE
SENSE
REPAIR

until GOAL

That is the notebook flow I'd want you to memorize.

---

11. Final conceptual chain

Write this at the bottom:

A∗:g+h→heuristic priority search\boxed{ A^* : g+h \rightarrow \text{heuristic priority search} } LPA∗:A∗+rhs+g/rhs consistency+reuse old search\boxed{ LPA^* : A^* + rhs + g/rhs\ consistency + reuse\ old\ search } D∗Lite:reverse LPA∗+goal-to-robot search+moving start+km\boxed{ D^*Lite : reverse\ LPA^* + goal\text{-}to\text{-}robot\ search + moving\ start + k_m }

and connect it backward to the previous session:

ABVI→selective backward updates→D∗Lite→formal selective backward repair with g/rhs\boxed{ ABVI \rightarrow \text{selective backward updates} \rightarrow D^*Lite \rightarrow \text{formal selective backward repair with }g/rhs }

Now your conceptual chain is complete:

Dijkstra
  ↓
A*
  ↓
LPA*
  ↓

D* Lite

and independently:

BVI
↓
ABVI
↓
same selective-repair intuition
↓
D* Lite

So D* Lite is where the **search side** and the **value-iteration side** of the course basically meet.